# Kubernetes and Istio

## 03 - Pod

Generated: June 2026
Version 1.0

# 3 - Pod

## Pod overview

A pod is a co-located group of containers and serves as the basic building block in Kubernetes. Instead of deploying containers individually, you deploy and manage this group of containers as a single, manageable unit. It represents a running instance of your application.

While containers are generally isolated, the processes running in a pod's containers share certain Linux namespaces to function together seamlessly. All containers in a pod share the **same Network namespace**, meaning they **share the same network interfaces, IP addresses, and port space** (eliminating port conflicts between different pods while allowing containers in the same pod to communicate via localhost). They also share the **UTS namespace** (seeing the same system hostname) and the **IPC namespace** (allowing standard inter-process communication).

**Isolated Mount Namespaces and Volumes**: It is important to note that containers in a pod do not share the mount namespace by default. Each container has its own isolated file system provided by its container image. If containers within the same pod need to share files, you must explicitly define a Kubernetes Volume and mount it into the required containers.

A fundamental architectural rule in Kubernetes is that a single pod instance never spans multiple nodes.
Because they share local resources, networking, and namespaces, all containers defined within a specific pod are strictly co-located and execute on the exact same physical or virtual worker node.
When you create a pod object, `kube-scheduler` (a scheduler managed by the Kubernetes control plane) schedules it to an appropriate worker node based on resource availability and requirements. Once scheduled, the Kubelet service on that specific node takes over to pull the required images and start the containers.

This is the minimal YAML template for defining a Pod:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
  labels:
    app: my-app # Optional, useful for selecting the pod afterward
spec:
  containers:
  - name: my-container
    image: nginx:alpine
```

## 1. Exploring Pods with `kubectl explain pod`

Before deploying objects, it is essential to understand their structure.

- You can explore the fields of a pod manifest and find out what additional fields can be added by using the kubectl explain pods command.
- This command acts as a built-in documentation tool, providing details about the API object's specifications and expected configurations.

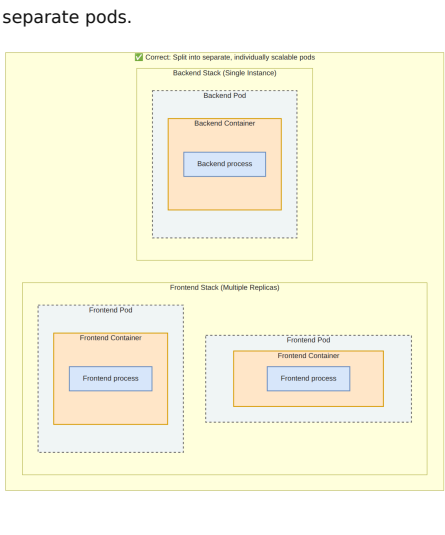
```
kubectl explain pod
kubectl explain pod.spec
...
```

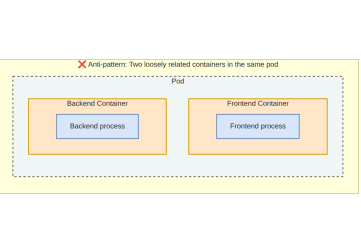
## 2. Structuring Pods for replication and Sidecar Pattern

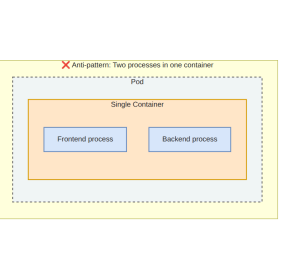
Pods are the basic building blocks in Kubernetes, consisting of a co-located group of containers.

- **Single Process per Container**: Containers are specifically designed to run only a single process. If you run multiple processes in one container, they all write to the same output, making logs intertwined and difficult to manage. Additionally, the container runtime only restarts a container when its root process dies, ignoring child processes.
- **Replication and Scaling**: A pod is the basic unit of both deployment and horizontal scaling. Kubernetes replicates the entire pod, not individual containers within it.

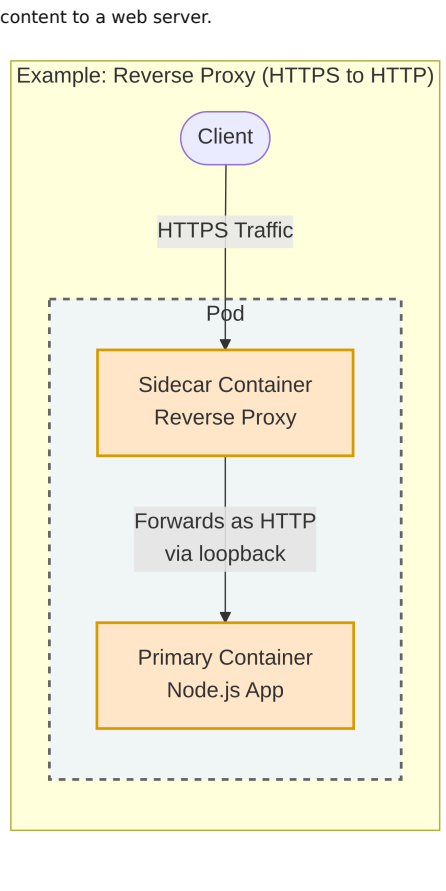
Therefore, components with different scaling requirements (e.g., a stateless frontend and a stateful backend) should be split into separate pods.

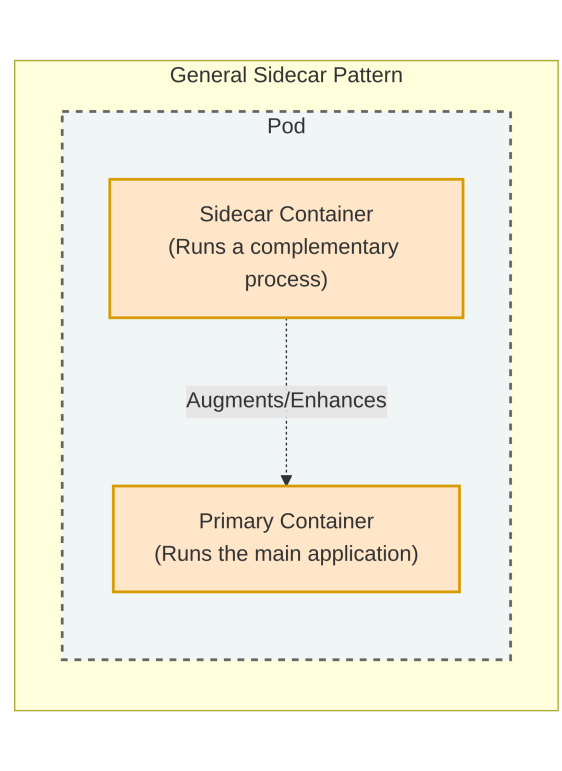

✅ Correct: Split into separate, individually scalable pods
Backend Stack (Single Instance)
Backend Pod
Backend Container
Backend process
Frontend Stack (Multiple Replicas)
Frontend Pod
Frontend Container
Frontend process
Frontend Pod
Frontend Container
Frontend process


❌ Anti-pattern: Two loosely related containers in the same pod
Pod
Backend Container
Backend process
Frontend Container
Frontend process


❌ Anti-pattern: Two processes in one container
Pod
Single Container
Frontend process
Backend process

- **Sidecar Containers**: You should place multiple containers in a single pod only if they form a unified whole and share resources.
  A "**sidecar container**" runs a complementary process to augment the primary application container. Examples include adding a reverse proxy for HTTPS support without modifying the main application's code, or an agent that delivers content to a web server.


Example: Reverse Proxy (HTTPS to HTTP)
Client
HTTPS Traffic
Pod
Sidecar Container
Reverse Proxy
Forwards as HTTP
via loopback
Primary Container
Node.js App


General Sidecar Pattern
Pod
Sidecar Container
(Runs a complementary process)
Augments/Enhances
Primary Container
(Runs the main application)

## 3. Debugging pods

During development or debugging, you may need to bypass load balancers and communicate directly with a specific pod.

- The `kubectl port-forward` command allows you to communicate with a pod through a proxy bound to a network port on your local computer.
- For example, executing `kubectl port-forward product-service 8080` forwards your local port 8080 to the pod's port 8080.
- The communication path routes from your local proxy to the Kubernetes API server, then to the Kubelet on the hosting node, and finally to the container through the pod's loopback device.

Moreover, containers typically log to the standard output and standard error streams. To display them you can simply run:

```
kubectl logs POD_NAME
```

Another Kubernetes useful debugging feature comes with the ability of copying files from pods to local computer and vice versa.

```
# Copy from pod
kubectl cp POD_NAME:html/index.html /tmp/index.html
# Copy to pod
kubectl cp /tmp/index.html POD_NAME:html/
```

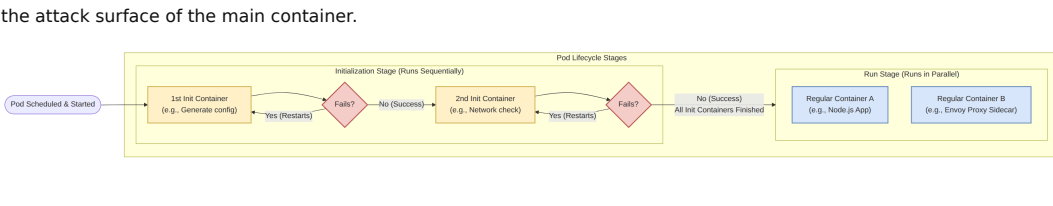
In a similar way to Docker CLI, it is possible to execute commands and attach to a remote pod:

```
# execute a command
kubectl exec POD_NAME -- COMMAND

# execute a command interactively (eg. bash)
kubectl exec -it POD_NAME -- bash

# attach to a pod
kubectl attach POD_NAME
```

## 4. Init Containers

Init containers are special containers intended strictly to initialize a pod before the regular containers start.

- **Execution Flow**: They execute sequentially. Each init container must finish successfully before the next one starts, and all must complete before the pod's main containers launch in parallel.
- **Use Cases**: They are used to initialize files in shared volumes, configure the pod's network system, or delay main container startup until a specific precondition (like an external database becoming available) is met.
- **Security**: By placing initialization logic requiring secret tokens or elevated privileges inside an init container, you shrink the attack surface of the main container.


Pod Scheduled & Started
Pod Lifecycle Stages
Initialization Stage (Runs Sequentially)
1st Init Container
(e.g., Generate config)
Fails?
Yes (Restarts)
No (Success)
2nd Init Container
(e.g., Network check)
Fails?
Yes (Restarts)
No (Success)
All Init Containers Finished
Run Stage (Runs in Parallel)
Regular Container A
(e.g., Node.js App)
Regular Container B
(e.g., Envoy Proxy Sidecar)

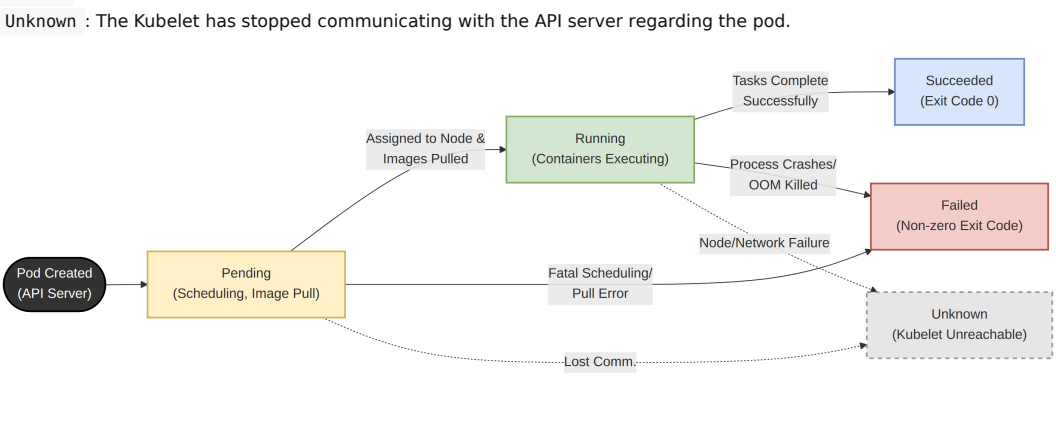
Here is an example of a Pod YAML configuration with an init container specified:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: minimal-init-demo
spec:
  initContainers:
  - name: init-task
    image: busybox:1.28
    command: ['ping','-c 1', '1.1.1.1']

  containers:
  - name: main-app
    image: nginx:alpine
    ports:
    - containerPort: 80
```

## 5. Pod phases

A pod goes through a distinct lifecycle, tracked by its phase.

- `Pending` : The initial phase where the pod is scheduled to a node and the container images are being pulled.
- `Running` : At least one of the pod's containers is actively running.
- `Succeeded` : All containers in the pod have terminated successfully (typically for finite tasks).
- `Failed` : At least one container terminated with a non-zero exit code.
- `Unknown` : The Kubelet has stopped communicating with the API server regarding the pod.


Pod Created
(API Server)
Pending
(Scheduling, Image Pull)
Assigned to Node &
Images Pulled
Running
(Containers Executing)
Tasks Complete
Successfully
Succeeded
(Exit Code 0)
Process Crashes/
OOM Killed
Failed
(Non-zero Exit Code)
Node/Network Failure
Fatal Scheduling/
Pull Error
Unknown
(Kubelet Unreachable)
Lost Comm.

You can examine in which status a pod is with the following command:

```
kubectl describe pod POD_NAME
```

## 6. Pod conditions

While phases offer a high-level summary, pod conditions specify whether a pod has reached critical milestones.

- `PodScheduled` : Indicates if the pod has been successfully assigned to a node.
- `Initialized` : Confirms that all init containers have run to completion successfully.
- `ContainersReady` : Indicates that all individual containers in the pod report being ready.
- `Ready` : Confirms the pod is fully ready to provide services to clients (both Containers and network are ready).
- Conditions can switch between `True` , `False` , or `Unknown` , and often provide a reason and message detailing their current status.

You can examine in which conditions a pod is with the following command:

```
kubectl describe pod POD_NAME
```

## 7. Container Status

Inside the pod's status, Kubernetes tracks the exact state of each individual container.

- `Waiting` : The container is waiting to start; the status includes a reason explaining the delay.
- `Running` : Processes are actively executing inside the container.
- `Terminated` : The container's processes have stopped; the status will show the specific exit code.

## 8. Pod Restart Policy

Kubernetes ensures self-healing by restarting containers based on the pod's restartPolicy.

- `Always` : The default policy. The container is restarted regardless of whether it exited successfully or failed.
- `OnFailure` : The container is restarted only if it terminates with a non-zero exit code.
- `Never` : The container is never restarted.
- `Exponential Back-off` : To prevent failing containers from constantly overloading the system, Kubernetes inserts an exponentially increasing delay before restarting a crashed container (10s, 20s, 40s, etc.), capped at 5 minutes.

## 9. Liveness Probes

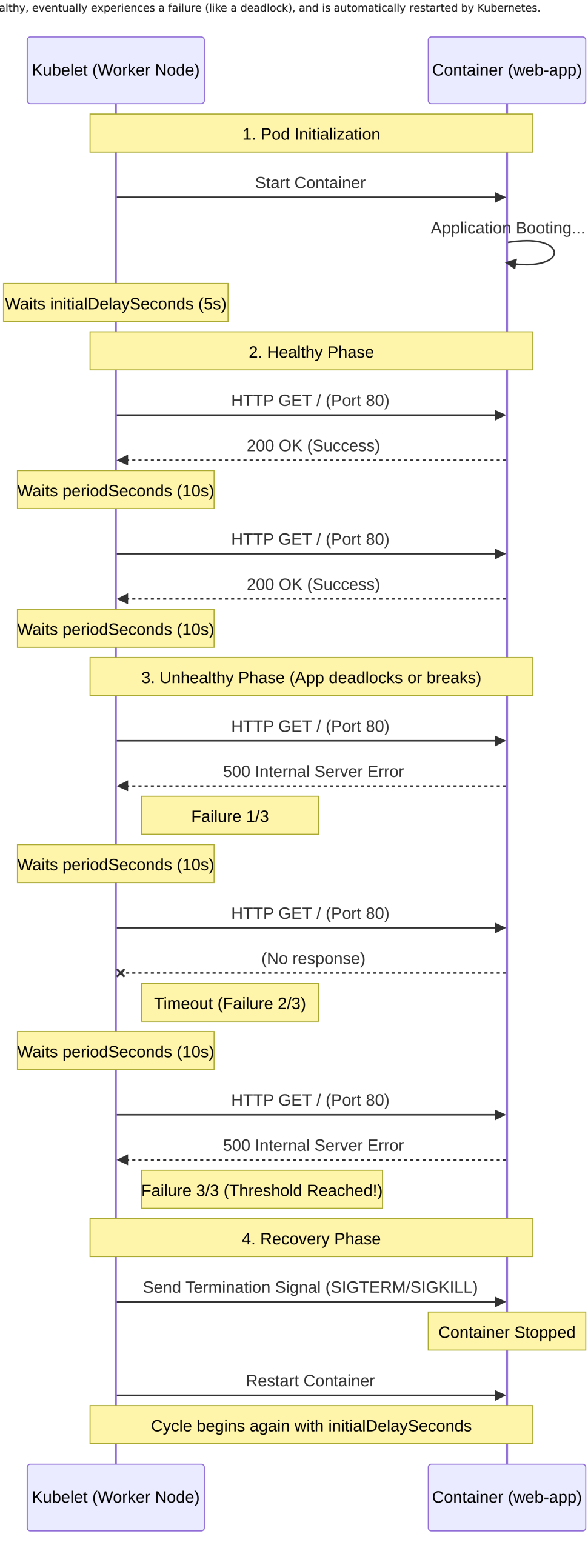
Applications can become unresponsive (e.g., deadlocks or infinite loops) without their processes actually terminating. Liveness probes allow Kubernetes to check the health of an application externally.

- **Mechanism**: If a liveness probe fails consecutively, the container is considered unhealthy, forcibly terminated, and restarted (subject to the restart policy).
- **Probe Types**:
  - **HTTP GET**: Considered successful if the server responds with a 2xx or 3xx HTTP status code.
  - **gRPC**: Considered successful if the application implements the gRPC health checking protocol and responds with a healthy status.
  - **TCP Socket**: Considered successful if a TCP connection can be established on a specified port.
  - **Exec**: Executes a command inside the container; successful if the command returns an exit code of zero.
- Configuration: Probes are customized using parameters like `initialDelaySeconds` (how long to wait before the first check), `periodSeconds` (interval between checks), `timeoutSeconds` (how long to wait for a response), and `failureThreshold` (number of consecutive failures required to trigger a restart).

In this example, we deploy a standard NGINX web server. We configure the liveness probe to send an HTTP GET request to the root path (/) on port 80.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: liveness-http-demo
spec:
  containers:
  - name: web-app
    image: nginx:alpine
    ports:
    - containerPort: 80

    # Define the liveness probe
    livenessProbe:
      httpGet:
        path: /
        port: 80
      # Timing and Threshold Configurations:
      initialDelaySeconds: 5  # Wait 5 seconds after container starts before first check
      periodSeconds: 10       # Perform the check every 10 seconds
      timeoutSeconds: 2       # Consider the check failed if no response within 2 seconds
      failureThreshold: 3     # Restart the container after 3 consecutive failures
```

This diagram illustrates the lifecycle of the liveness probe defined in the YAML above. It shows a scenario where the application starts healthy, eventually experiences a failure (like a deadlock), and is automatically restarted by Kubernetes.


Kubelet (Worker Node)
Container (web-app)
1. Pod Initialization
Start Container
Application Booting...
Waits initialDelaySeconds (5s)
2. Healthy Phase
HTTP GET / (Port 80)
200 OK (Success)
Waits periodSeconds (10s)
HTTP GET / (Port 80)
200 OK (Success)
Waits periodSeconds (10s)
3. Unhealthy Phase (App deadlocks or breaks)
HTTP GET / (Port 80)
500 Internal Server Error
Failure 1/3
Waits periodSeconds (10s)
HTTP GET / (Port 80)
(No response)
Timeout (Failure 2/3)
Waits periodSeconds (10s)
HTTP GET / (Port 80)
500 Internal Server Error
Failure 3/3 (Threshold Reached!)
4. Recovery Phase
Send Termination Signal (SIGTERM/SIGKILL)
Container Stopped
Restart Container
Cycle begins again with initialDelaySeconds
Kubelet (Worker Node)
Container (web-app)